Assignment 2

STA410, Winter 2026

Assigned on February 27, 2026.
Submit by 11:59 pm on March 12, 2026 on Qeurcus.

1. (**Avoiding Round-Off Error**) In this problem, we will consider two algorithms for computing the sample variances that avoid catastrophic cancellation. Both are "one pass" algorithms, in the sense that we only need to cycle once through the data; the standard variance expression of n samples $x^{(1)}, \ldots, x^{(n)}$ given by

$$\frac{1}{n-1}\sum_{i=1}^{n}(x^{(i)} - \bar{x})^2$$

needs two passes through the data since we need to first compute the sample mean, $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x^{(i)}$ before computing the sum of the squared differences. In parts (a) and (b) below, define $\bar{x}_k$ to be the sample mean of the first k samples $x^{(1)}, \ldots, x^{(k)}$ and note that

$$\bar{x}_{k+1} = \frac{k}{k+1}\bar{x}_k + \frac{1}{k+1}x^{(k+1)},$$

with $\bar{x}_n = \bar{x}$.

(a) Show that $\sum_{i=1}^{n}(x^{(i)} - \bar{x})^2$ can be computed using the recursion

$$\sum_{i=1}^{k+1}(x^{(i)} - \bar{x}_{k+1})^2 = \sum_{i=1}^{k}(x^{(i)} - \bar{x}_k)^2 + \frac{k}{k+1}(x^{(k+1)} - \bar{x}_k)^2,$$

for $k = 1, \ldots, n-1$. This is known as West's algorithm.

(b) A somewhat simpler one-pass method replaces $\bar{x}$ by some estimate x_0 and then corrects for the error in estimation. Specifically, if x_0 is an arbitrary number, show that

$$\sum_{i=1}^{n}(x^{(i)} - \bar{x})^2 = \sum_{i=1}^{n}(x^{(i)} - x_0)^2 - n(x_0 - \bar{x})^2.$$

(c) The key in using the formula in part (b) is to choose x_0 to avoid catastrophic cancellation, that is, x_0 should be close to $\bar{x}$. One possibility is take x_0 to be the average of a subset of $m \ll n$ observations. What considerations should be used in choosing m? Implement this for data samples $x^{(i)} \sim \mathcal{N}(10^8, 1)$ and report your observations.

2. (**Iterative Algorithms**) Suppose we observe $y^{(1)}, \ldots, y^{(N)}$ where $y^{(i)} = \theta_i + \epsilon^{(i)}$ for $i = 1, \ldots, n$ where $\epsilon^{(i)}$ is a sequence of random variables with mean 0 and finite variance representing noise. We will assume that $\theta_1, \ldots, \theta_n$ are smooth in the sense that $\theta_i = g(i)$ for some continuous function g. In this problem, we will consider estimating $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_n)$ by solving the minimization problem

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n}(y^{(i)} - \theta_i)^2 + \lambda \sum_{i=2}^{n-1}(\theta_{i+1} - 2\theta_i + \theta_{i-1})^2,$$

where $\lambda > 0$ is a tuning parameter that controls the smoothness of the estimates $\widehat{\theta}_1, \ldots, \widehat{\theta}_n$

(a) Show that if $y^{(i)}$ are exactly linear, i.e., $y^{(i)} = ai + b$ for some a, b, then $\widehat{\theta}_i = y^{(i)}$ for all i.

(b) In principle, we would compute $\widehat{\theta}_1, \ldots, \widehat{\theta}_n$ using ordinary least squares esitmation. Show that $\widehat{\boldsymbol{\theta}} = (\widehat{\theta}_1, \ldots, \widehat{\theta}_n)^T$ solve

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^n}{\arg\min} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2,$$

What are $\mathbf{y} \in \mathbb{R}^{2n-2}$ and $\mathbf{X} \in \mathbb{R}^{(2n-2)\times n}$?

(c) When n is large, computing $\widehat{\theta}_1, \ldots, \widehat{\theta}_d$ directly, for example, using the formulation in part (b) is computationally expensive. Alternatively, we could use the Gauss-Seidel algorithm, particularly for larger values of λ, but this can converge slowly. One possible alternative is a randomized modification of the Gauss-Seidel algorithm, which at each stage solves a least squares problem with $p \ll n$ variables. The basic algorithm is as follows:

1. Initialize $\widehat{\boldsymbol{\theta}}$;
2. Randomly sample a subet w of size p from the integers $1, \ldots, n$. Define X_w to be the submatrix of $\mathbf{X}$ with column indices w and $\mathbf{X}_{\bar{w}}$ to be the submatrix of $\mathbf{X}$ with colum indices in the complement of w; define $\boldsymbol{\theta}_w$ and $\boldsymbol{\theta}_{\bar{w}}$ analogoously so that $\mathbf{X}\boldsymbol{\theta} = \mathbf{X}_w\boldsymbol{\theta}_w + \mathbf{X}_{\bar{w}}\boldsymbol{\theta}_{\bar{w}}$;
3. Define $\widehat{\boldsymbol{\theta}}_w$ to minimize
$$\|\mathbf{y} - \mathbf{X}_w\boldsymbol{\theta}_w + \mathbf{X}_{\bar{w}}\widehat{\boldsymbol{\theta}}_{\bar{w}}\|^2$$
with respect to $\boldsymbol{\theta}_w$ where $\widehat{\boldsymbol{\theta}}_{\bar{w}}$ is fixed;
4. Repeat Steps 2 and 3 until convergence.

This is known as the randomized block Gauss-Seidel algorithm. Implement the algorithm using a function that takes in λ, p and `niter`, which specifies the number of iterations of the algorithm. The output of the function consists of two components: the estimates of θ_i and the values of the objective function for each iteration.

(d) Use the function from part (c) with $\lambda = 2000$ on data from monthly yields of short-term British securities over a 21 year period (252 months). The data $\mathbf{y}$ is contained in the file `yield.txt` on Quercus (Hint: Format the data as a vector first). Try out various values of p between 5 and 50 using 1000 iterations. Describe how the objective function decreases with each iteration as p varies between 5 and 50.

3. **(Efficiency of Solving Linear Systems)** In this problem we will compare the efficiency of different linear algebra methods for numerically solving an ordinary differential equation (ODE) in a finite one-dimensional domain $[0, L]$ of length L. We will take $u(x)$ to be the concentration of a pollutant in a river section of length L. The river flows at speed a. The pollutant is leaked from pipes according to the source function $p(x)$ and the following ODE governs the distribution of pollutant:

$$0 = -a\frac{du}{dx} + D\frac{d^2u}{dx^2} + p(x).$$

subject to the boundary conditions: $u(0) = u(L) = 0$. We will take the parameters of our problem to be $L = 10, a = 1, D = 0.1$ and p is given by

$$p(x) = \begin{cases} 0.01(\cos(2\pi x + \pi) + 1) & 0 \le x \le 3 \\ 0 & \text{otherwise} \end{cases}.$$

To solve the ODE we start by discretizing the domain $[0, L]$ into a grid of $N + 1$ equally spaced points

$$0 = x_0 < x_1 < \cdots < x_{N-1} < x_N = L,$$

where $x_i = iL/N$ and $\Delta x = L/N$. We will solve the equation at these grid points $\{u(x_i)\}_{i=0}^{N}$. We will use these grid values to approximate the values of the derivatives of u at the interior points of the domain. We will use the following finite difference approximations of the derivative:

$$\frac{du}{dx}(x_i) \approx \frac{u(x_{i+1}) - u(x_{i-1})}{2\Delta x}, \quad \text{for } i = 1, \cdots, N-1$$
$$\frac{d^2u}{dx^2}(x_i) \approx \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1})}{(\Delta x)^2}, \quad \text{for } i = 1, \cdots, N-1$$

Note that here $u(0)$ and $u(L)$ are fixed at 0 by the boundary conditions.

(a) Given the known boundary conditions at the first and last grid point, we have $N-1$ unknown values of the pollutant at the interior grid points. Substituting the finite differences into the ODE, write a system of $N-1$ equations for the solution on the interior grid points. Then express the solution for the vector $\mathbf{u} = [u(x_1), \ldots, u(x_{N-1})]^T$ as the solution to a matrix system

$$\mathbf{A}\mathbf{u} = -\mathbf{p},$$

for some matrix $\mathbf{A} \in \mathbb{R}^{N-1 \times N-1}$ and $\mathbf{p} \in \mathbb{R}^{N-1}$ where $\mathbf{p}_i = p(x_i)$ is the pollutant source function evaluated at these grid points.

(b) Write a function that takes the number of grid points N as input and returns matrix $\mathbf{A}$ and $\mathbf{p}$.

(c) The matrix in this application has tridiagonal structure. To see this, you can use the command `matplotlib.pyplot.spy` to visualize the sparsity pattern of $\mathbf{A}$. This command shows where in the matrix there are non-zero entries. In class, we discussed several different algorithms that can be used to solve a linear system, depending on the structure of: LU decomposition, Cholesky decomposition, tridiagonal matrix algorithm (TDMA), and forward/backward substitution. Which of these algorithms are applicable here and why? Hint: To check to see if a matrix is symmetric, you can look at $\mathbf{A} - \mathbf{A}^T$.

(d) While `np.linalg.solve` is generally what you should use to solve linear systems, in this problem we'd like to explore the cost scaling of the different linear solve algorithms. To that end, compare the cost of Python's solve function to the cost of solving the linear system with the TDMA algorithm mentioned in class (see the provided code `tdma.py`), and the cost to computing $\mathbf{A}^{-1}\mathbf{p}$, even though this is not a good idea. For each run, the times may change slightly, so it is often best to repeat these cost calculations multiple times and average the results. (Hint: You may find `time.time` useful to evaluate the runtime of a function in Python. Exact run times will vary from machine to machine.)

(e) At large N from 100 to 1600, how does the cost of each of the solve types change as the matrix size is doubled? How does the expected cost scaling compare to the expected big-N property we discussed in class for each method.